

**Université Mohamed Premier
Faculté Pluridisciplinaire
Département de Mathématiques & Informatique
◊ Nador ◊**

Support de Cours Intelligence Artificielle II

◊ Master : Sciences des Données et Systèmes Intelligents- S2

**Réalisé par:
Pr. Mohamed Bellouki**

◊ Année Universitaire 2019-2020

Sommaire

Chapitre 1	Systèmes Experts et Moteur d'Inférence
Chapitre 2	Contraintes et Problèmes de Satisfaction de Contrainte
Chapitre 3	Modélisation de CSPs
Chapitre 4	Résolution de CSPs
Chapitre 5	Réalisation de solveurs de contraintes

1.1 Introduction

Un système expert est un logiciel destiné à remplacer ou assister l'homme dans des domaines où est demandée une expertise humaine considérable. Il permet l'expression facile et directe de la connaissance logique, et l'exploitation de cette connaissance avec logique au cours d'un dialogue avec l'utilisateur. Il est capable de faire un enchaînement de déductions permettant ainsi d'inférer ou de produire de nouvelles connaissances. Le système expert peut manipuler indifféremment des connaissances de type varié pouvant être des idées, des jugements, des décisions, des propositions, des prédictions, etc. Par ailleurs, un système expert supporte facilement les révisions, les ajouts et les suppressions de faits ou de règles.

Un système Expert est utile pour faire des tâches d'analyse, de synthèse, etc. Il aura pour principe d'organiser et de diffuser une connaissance par des classifications, diagnostics, interprétations, analyses d'objets, configurations, conceptions, planifications, commandes, contrôles, prédictions, réparations. Il sera une aide pour identifier des menaces, des risques, des pannes, des maladies, des actions. Le premier système expert fut Dendral en 1965, créé par les informaticiens Edward Feigenbaum, Bruce Buchanan, le médecin Joshua Lederberg et le chimiste Carl Djerassi.

1.2 Exemples et évolution des Systèmes Experts

Les principes d'un système expert ont subi une grande évolution depuis leur création. Ainsi l'apparition des premiers SE capable de s'exprimer en langage naturel et aussi d'expliquer leur raisonnement lors de la réponse à une question utilisateur. Ce type de SE dit de deuxième génération, dotés de capacité d'apprentissage.

Une liste de quelques systèmes experts les plus connus dans différents domaines.

SE	discipline	Année	Lieu de développement
DENDRAL	Chimie	1969	Université Stanford, USA
MYCIN	Médecine	1972-1974	Université Stanford, USA
PROSPECTER	Géologie	1978	S. R. I Stanford, USA
DART	Informatique	1981	IBM, USA
LPS	Maths	1979	Université Kcio, Japan
PEGANE	Gestion	1982	Ecole polytech, Lousanne
SONEX	Tr. Parole	1984	Labo, LIMSI, Univ, Paris11

1.3 Composants d'Système expert

1.3.1 Architecture de base

Un système expert est composé de deux parties complémentaires :

- 1) Une base de connaissances : qui contient l'ensemble de la connaissance acquise dans un domaine d'application. Cette base de connaissance est constituée elle-même de deux parties

-une *Base de Faits* (BDF) : qui contient la séquence de faits établis et ayant une valeur de vérité vraie. Elle constitue la partie pratique de la base de connaissances.

-une *base de règles* (BDR) : qui contient l'ensemble des règles de production pouvant être appliquées aux faits pour déduire de nouveaux faits non existant préalablement dans la BDF. IL s'agit de la partie dynamique et fondamentale de la base de connaissance. En effet, un SE peut démarrer son raisonnement avec une BDF vide, mais il ne peut faire aucun raisonnement si sa BDR ne contient aucune règle.

*La base de connaissance constitue *la mémoire d'un système expert*.

- 2) *Un moteur d'Inférence (MI)* : il s'agit du cerveau du système expert. C'est un programme ou procédure qui simule le raisonnement humain par des applications répétitives de l'expert, les règles, aux faits d'un problème pour déduire de nouveaux faits ou résultats. Au cours du traitement, le moteur d'inférence peut générer plusieurs faits intermédiaires qui serviront de base pour l'application de nouvelles règles et ainsi jusqu'à l'aboutissement au résultat cherché ou à une impasse.

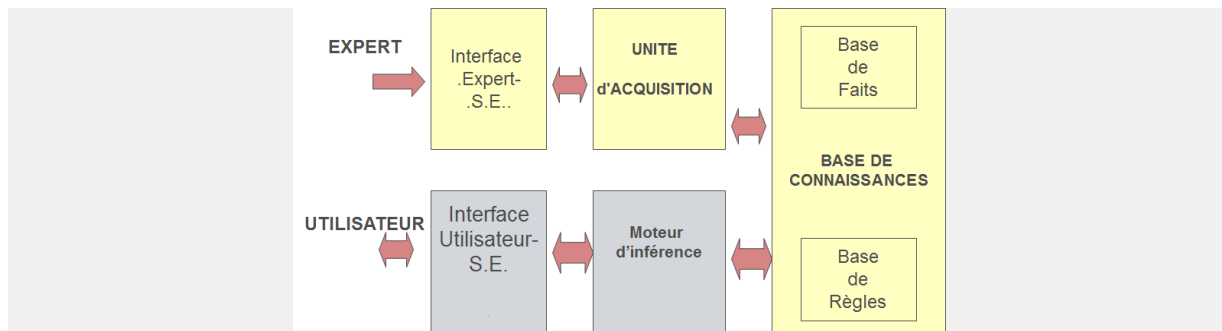
En générale le moteur d'inférence est une partie du SE qui est indépendante de la connaissance. Ainsi, un même moteur d'inférence peut être appliqué à des bases de connaissances variées pour constituer à chaque fois un SE différent. C'est le cas de moteurs tels que le langage Prolog qui possède au sein de lui-même un moteur d'inférence, ce qui explique sa capacité de déduction est d'adaptation à différentes bases de connaissances.

1.3.2 Eléments complémentaires d'un SE

Un système expert doit contenir en général des modules supplémentaires pour faciliter la tâche de compréhension à l'utilisateur et une capacité d'évolution, on peut citer :

- *un module d'acquisition de la connaissance* : pour faciliter l'évolution du SE.
- *Un module de justification* : pour expliquer son raisonnement à l'utilisateur.
- *Une interface utilisateur* : à l'aide du langage naturel comme support de communication avec l'utilisateur.

L'architecture complète d'un système expert est donnée comme suit :



1.4 Réalisation d'un système expert

1.4.1 Phase de conception

Comme pour le développement de tout logiciel, il est nécessaire de passer par une phase de conception pendant laquelle on dégage tous les éléments de l'application et le raccordement logique entre ces éléments. Pendant cette phase on détermine aussi les besoins et le type d'entrées/sorties nécessaire à l'application ; type de problème peut rencontrer l'utilisateur, comment formuler ses besoins à l'utilisateur, genre de réponse souhaite obtenir. Nous pouvons donner un plan d'actions sous forme d'un ensemble d'étapes à réaliser. Cependant nous ne pouvons pas prétendre qu'il existe une démarche rigoureuse est finie comme le cas de Merise appliquée aux systèmes de gestion classiques. Mais l'ingénieur de la connaissance peut mener sa démarche en s'inspirant d'une méthode d'analyse classique ou moderne qq qu'il applique au domaine de l'expertise étudiée en passant par les techniques de l'informatique symbolique.

La phase de conception d'un système expert peut être définie par les étapes suivantes :

- 1) Etablir un contact avec un ou plusieurs experts dans le domaine.
- 2) Etablir la liste des données logiques disponibles ;
- 3) € Etablir la liste des données à déduire et l'ensemble des règles de déduction.
- 4) Associer à chaque règle un degré de certitude dans le cas ou les règles de déduction n'offrent pas des résultats surs.
- 5) Préparation de la base de connaissances équivalente : choix et rassemblement des prédicats et représentation symbolique ou graphique de l'ensemble des règles.
- 6) Définition de l'interface utilisateur : sou le terme d'interface, on regroupe à la fois le moyen de communication (entrées/sorties en langage naturel) et la représentation des boites de dialogue.

1.4.2 Phase de d'implantation

La partie la plus intéressante dans système expert est son noyau (BDC+ MI). Pour développer un système expert, il est indispensable d'utiliser un système informatique

spécifique qui offre un moyen souple de représentation de la connaissance. Deux choix se présentent :

- 1) Un premier choix consiste à utiliser un langage disposant un moteur d'inférence indépendant. On peut alors utiliser le langage Prolog qui a donné de très bons résultats avec de nombreuses applications.
- 2) Un deuxième choix consiste à utiliser un langage spécifique à l'intelligence artificielle avec lequel on peut faire un choix du moteur d'inférence à utiliser et de la méthode de représentation de la connaissance. Dans ce cas, le programmeur doit réaliser en premier lieu un moteur d'inférence ensuite une implantation adéquate de la base de connaissance.

1.5 Structure et caractéristiques d'un moteur d'inférence

1.5.1 Cycles d'un moteur d'inférence

Le moteur d'inférence une séquence de cycles au cours de son raisonnement jusqu'à aboutir au résultat désiré ou jusqu'à saturation. Un cycle du moteur d'inférence est composé de deux phases : *une phase d'évaluation et une phase d'exécution.*

1.5.1.1 Phase d'évaluation

Pendant cette phase, le moteur d'inférence essaye de dégager l'ensemble de règles qui peuvent tirées ou déclenché à la phase suivante. Cette phase d'évaluation est effectuée en 3 étapes :

- 1) *La sélection ou restriction* : c'est une étape optionnelle pendant laquelle le moteur détermine un sous ensemble de règles et de faits dans lequel il est probable de trouver une solution. Dans ce cas, la BDC est supposée être décomposée en sous ensembles ou groupes de domaines distincts. Ainsi, pour la réponse à une requête utilisateur dans un domaine particulier, il est suffisant de faire une recherche uniquement dans le groupe correspondant.
- 2) *Filtrage ou génération de conflits* : dans cette étape, on fait la sélection des règles qui peuvent effectivement être déclenchées. Example : les règles dont les prémisses appartiennent à la base de faits. On obtient un sous ensemble de règles appelé *ensemble de conflits*.
- 3) *Résolution de conflits* : Dans cette, on fait le tri des règles déjà sélectionnées, on choisit la première règle (ou les premières règles) à déclencher.

1.5.1.2 Phase d'exécution

Dans cette phase on déclenche les règles dernièrement sélectionnées. Les nouveaux faits résultats seront ajoutés à la base de faits. Dans ce cas, si le but est atteint la recherche est

arrêtée, sinon, on recommence un nouveau cycle. Généralement cette phase consiste à ajouter des faits à la BDF, mais il est aussi possible d'en retirer d'autres ou de déclencher des réactions ou exécuter des tâches supplémentaires.

Remarque

Le traitement du MI peut aussi être arrêté lors du cycle de recherche dans le cas où la génération de conflits abouti à un ensemble vide.

1.5.2 Caractéristiques d'un moteur d'inférence

Dans ce paragraphe, on représente un ensemble de critères qui peuvent distinguer entre les moteurs d'inférence. La programmation d'un moteur d'inférence nécessite la sélection d'un ensemble de critères à réunir dans celui-ci. Les critères de base d'un moteur d'inférence sont : son mode d'invocation des règles, la stratégie de recherche qu'il utilise, son régime de contrôle et le critère de monotonie. Ainsi, en choisissant à chaque fois un mode d'invocation des règles donné, une stratégie de recherche, un régime de contrôle et en fixant la monotonie, on obtient un moteur d'inférence complet et bien définie.

1.5.2.1 Modes d'invocation des règles

Les MI sont distingués par leur mode d'invocation des règles. On peut classer les MI en 3 catégories :

- 1) *Les MI à chainage avant* : Un MI à chainage avant détermine le résultat (but) à partir de la BDF, les règles à déclencher à chaque cycle sont celles dont les prémisses appartiennent à la BDF. L'exécution de ces règles modifie la BDF et d'autres règles peuvent alors être déclenchées au cycle suivant. Ce principe sera répété jusqu'à ce que le but soit dans la BDF.

Exemple:

soit $BDF = \{A, B, C, D\}$

et soit la règle : $A, C \rightarrow G$

cette règle est déclenchable et on obtient $BDF = \{A, B, C, D, G\}$

L'étape de génération de conflit correspond à rassembler toutes les règles dont les prémisses sont dans la BDF.

- 2) *Les MI à chainage arrière* : Un MI à chainage arrière détermine l'ensemble des règles qu'il faut invoquer pour aboutir au but (fait à établir). Le but sera alors remplacé par les prémisses qui vont constituer les nouveaux faits à établir. Les règles à tirer à chaque cycle sont alors celles dont les conclusions sont égales au (contiennent le) fait à établir. L'exécution de ces règles ne modifie pas la BDF mais elle remplace le

problème initial par d'autres. Ce type de raisonnement correspond à une décomposition de problème en un ensemble de sous problèmes.

Exemple :

Soit le but à établir P, et soit la règle :

$A, B, C \rightarrow P$

Pour résoudre le problème P, il suffit de résoudre les sous problèmes A, B, C.

Ce mécanisme sera répété jusqu'à ce que la dernière liste de fait à établir soit entièrement dans la BDF.

L'étape de génération de conflit consiste à rassembler toutes les règles dont les conclusions contiennent le fait à établir.

- 3) *Les MI à chainage mixte* : Ce type de moteur peut être choisi lorsqu'une partie des faits du problème est à établir, l'autre est considérée déjà établie. Les conditions de déclenchement des règles dans ce cas peuvent porter sur les deux types de faits. Donc, résoudre le problème on sera amené à prendre en considération les faits déjà établis et de remplacer le problème à résoudre pour une liste de sous problèmes. Ceci, correspond à un mode de raisonnement mixte faisant intervenir simultanément le chainage avant et le chainage arrière.

1.5.2.2 Stratégies de recherche

Au cours des cycles de recherche d'un moteur d'inférence, on développe un arbre de recherche dans lequel chaque niveau correspond à l'ensemble des règles déclenchées (ensemble de conflits). Chaque règle déclenchée crée une nouvelle situation et de nouvelles règles à invoquer. Deux stratégies de recherche de la solution se présentent :

- Soit on développe toutes les règles d'un même niveau de l'arbre l'une après l'autre avant de passer au niveau suivant. Cette stratégie correspond à une recherche en largeur d'abord.
- Soit on passe d'un niveau à un autre à chaque fois qu'on déclenche une règle et ne revient aux règles restantes que si on épuise toutes les règles en profondeur sans avoir trouver la solution. Ceci correspond à une stratégie de recherche en profondeur d'abord. Le retour arrière dans le cas où la recherche en profondeur échoue est appelé : *backtracking*.

Remarques :

- 1) Un moteur d'inférence à chainage avant ou arrière peut être en profondeur ou en largeur d'abord.

- 2) Le mode d'invocation des règles et la stratégie de recherche constituent les critères de base pour la réalisation d'un moteur d'inférence.

1.5.2.3 Régime de contrôle

Le régime de contrôle pour un moteur d'inférence peut être irrévocable ou par tentative. Dans le cas d'un régime irrévocable, l'application d'une règle dans un cycle du moteur d'inférence n'est jamais remise en cause et on n'opère pas de backtracking. S'il n'y a plus de règles à appliquer le moteur d'inférence s'arrête et signale un échec sans faire de retour-arrière. Les régimes de contrôle par tentatives peuvent remettre en cause des règles déjà appliquées si elles n'ont pas abouti, et faire un backtracking en retirant aussi les faits qui en étaient déduits.

1.5.2.4 Monotonie

Un moteur d'inférence peut être monotone ou non-monotone. Un MI monotone garde chaque fait initial ou ajouté à la BDF jusqu'à la fin de la recherche, et tout fait ajouté n'introduit pas de contradiction avec les autres faits de la BDF. Un moteur d'inférence non monotone, peut retirer de la BDF un fait précédemment utilisé. Cette situation peut être rencontrée en cas de backtracking par exemple. Les faits déduits d'une règle qui n'a pas abouti seront supprimés à l'occasion de la remise en cause de la règle en question et le choix d'une autre en faisant un retour-arrière. Certains faits peuvent aussi être retirés de la base de faits s'ils présentent des contradictions avec d'autres.

La monotonie concerne aussi les règles. Un système monotone n'élimine jamais une règle de la BDF, toutes les règles sont considérées correctes jusqu'à la fin du raisonnement. Cependant un système non monotone peut retirer des règles en constatant des contradictions avec d'autres.

Remarque

Il ne faut pas confondre la propriété de monotonie et la propriété irrévocable. En effet, dans le cas d'un système monotone, on peut faire un retour-arrière mais les faits précédemment déduits ne sont pas retirés de la BDF. Cependant un système irrévocable ne retire pas les faits précédemment déduits car il ne remet jamais en cause une règle déjà appliquée et ne fait pas de retour-arrière.

1.6 Types de moteur d'inférence

1.6.1 MI à Chainage Avant

MI à chainage avant en profondeur d'abord monotone avec régime irrévocable

Exercice 1

Soit la base de connaissance symbolique suivante :

$BDF = \{A, D, E, G\}$

$BDR = \{ R1: A, B, C \rightarrow H$

$R2: A, U, C \rightarrow F$

$R3: E, G, B \rightarrow S$

$R4: D, G \rightarrow C$

$R5: A, E \rightarrow B$

$R6: U, S, T \rightarrow F$

$R7: G, H \rightarrow R$

$R8: D, E \rightarrow T$

$R9: R, S, H \rightarrow F$

$R10: A, U \rightarrow B \}$.

Problème: établir le fait F.

Solution

Cycle1 :

Conflits= $\{R4, R5, R8\}$,

On applique R4

$\rightarrow BDF = \{A, D, E, G, \underline{C}\}$.

Cycle2 :

Conflits= $\{ R5, R8\}$, R4 déjà appliquée.

On applique R5

$\rightarrow BDF = \{A, D, E, G, C, \underline{B}\}$

Cycle3 :

Conflits= $\{ R1, R3, R8\}$

On applique R1

$\rightarrow BDF = \{A, D, E, G, C, B, \underline{H}\}$

Cycle4 :

Conflits= $\{ R3, R7, R8\}$

On applique R3

$\rightarrow BDF = \{A, D, E, G, C, B, H, \underline{S}\}$

Cycle5 :

Conflits= $\{R7, R8\}$

On applique R7

$\rightarrow BDF = \{A, D, E, G, C, B, H, S, \underline{R}\}$

Cycle6 :

Conflits={R8, R9}

On applique R8

→ BDF={A, D, E, G, C, B, H, S, R, T}

Cycle7 :

Conflits={R9}

On applique R9

→ BDF={A, D, E, G, C, B, H, S, R, T, E}

→ Succès.

Le raisonnement de MI peut être schématisé par un arbre qui illustre l'exploration en profondeur de l'arbre de recherche.

MI à chaînage avant en largeur d'abord monotone avec régime irrévocable

Exercice 2

On considère la même base de connaissances, avec le même fait F à établir

BDF = {A, D, E, G}.

Solution :

Cycle1 :

Conflits={R4, R5, R8},

On applique R4, puis R5, ensuite R8.

→ BDF={A, D, E, G, C, B, T}.

Cycle2 :

Conflits={R1, R3},

On applique R1, puis R3

→ BDF={A, D, E, G, C, B, T, H, S}.

Cycle3 :

Conflits={R7},

On applique R7

→ BDF={A, D, E, G, C, B, T, H, S, R}.

Cycle4 :

Conflits={R9},

On applique R9

→ BDF={A, D, E, G, C, B, T, H, S, R, E}.

→ Succès.

1.6.2 MI à Chainage Arrière

✚ MI à chainage avant en profondeur d'abord monotone avec régime par tentative

Le raisonnement de ce moteur d'inférence correspond au chemin inverse effectué par un moteur d'inférence à chainage avant. En effet, dans ce cas un fait à établir sera à chaque étape remplacé par d'autres faits qui sont les prémisses d'une règle dont le fait à établir est conclusion. La résolution des conflits consiste alors à déterminer l'ensemble des règles telles que le fait appartient à la conclusion. Les règles seront déclenchées l'une après l'autre sachant que le déclenchement d'une règle débute un nouveau cycle dont lequel il faudrait encore chercher un nouvel ensemble de conflits. C'est le cas en profondeur d'abord.

Exercice 3

Reprenons le même exercice, on considère la liste des problèmes $LDP = \{F\}$.

Cycle1 :

Conflits= $\{R2, R6, R9\}$,

On déclenche R2.

→ $LDP = \{\underline{A}, U, C\}$. Le fait souligné sera le prochain fait à établir.

Cycle2 :

$A \in BDF \rightarrow LDP = \{\underline{U}, C\}$.

Cycle3 :

Pour résoudre U, l'ensemble de Conflits= $\{\}$

→ Echec.

→ backtracking aux conflits de cycle 1 ;

→ $LDP = \{F\}$.

Cycle4 :

Conflits= $\{R6, R9\}$,

On déclenche R6.

→ $LDP = \{\underline{U}, S, T\}$.

Cycle5 :

Pour résoudre U, l'ensemble de Conflits= $\{\}$

→ Echec.

→ backtracking aux conflits de cycle 1 ;

→ $LDP = \{F\}$.

Cycle6 :

Conflits={R9},

On déclenche R9.

→ LDP={R, S, H}.

Cycle7 :

Conflits={R7},

On déclenche R7.

→ LDP={G, H, S, H}={G,S, H}

Cycle8 :

G ∈ BDF → LDP={S, H}

Cycle9 :

Conflits={R3},

On déclenche R3 → LDP={E, G, B, H}

Cycle10 :

E ∈ BDF → LDP={G, B, H}

Cycle11 :

G ∈ BDF → LDP={B, H}

Cycle12 :

Conflits={R3, R10},

On déclenche R5 → LDP={A, E, H}

Cycle13 :

A ∈ BDF → LDP={E, H}

Cycle14 :

E ∈ BDF → LDP={H}

Cycle15 :

Conflits={R1},

On déclenche R1 → LDP={A, B, C}

Cycle16 :

A ∈ BDF → LDP={B, C}

Cycle17 :

Conflits={R5, R10},

On déclenche R5 → LDP={A, E, C}

Cycle18 :

A ∈ BDF → LDP={E, C}

Cycle19 :

$E \in BDF \rightarrow LDP = \{\underline{C}\}$

Cycle20 :

Conflits = {R4},

On déclenche R4 $\rightarrow LDP = \{\underline{D}, G\}$

Cycle21 :

$D \in BDF \rightarrow LDP = \{\underline{G}\}$

Cycle22 :

$G \in BDF \rightarrow LDP = \{\}$

$\rightarrow$ Succès.

Remarque

Ce raisonnement peut être représenté par un graphe dont lequel on effectue une recherche en profondeur. Un cercle représentera un fait à établir, et un rectangle représente un fait établi.

1.6.3 MI à Chainage Mixte

Le système peut raisonner à partir des faits dont il dispose (chainage avant) aussi bien qu'à partir des buts à atteindre (chainage arrière). Les règles font appel simultanément à des faits établis et d'autres à établir.

Exemple de règles dans ce cas : $P, F :-Q$

Qui se traduit : pour résoudre le problème P, sachant qu'on a F, il suffit de résoudre le problème Q. le raisonnement utilise :

-un chainage avant : la règle ne peut être déclenché que si $F \in BDF$, F constitue se qu'on appelle un Filtre. En absence de ce filtre, le moteur d'inférence fonctionne alors en chainage arrière.

-un chainage arrière : le problème P (conclusion de la règle) est remplacé par le problème Q (prémisse de la règle).

Ce type de règles peut être généralisé en introduisant des possibilités d'ajouter ou de retrancher des faits au cours de raisonnement (non monotonie). Ceci est effectué à l'aide de la notion d'actes. Ces derniers sont des actions élémentaires qui permettent d'ajouter ou de retrancher des faits.

Chapitre 2 Contraintes et Problèmes à Satisfaction de Contraintes

2.1 Introduction

De nombreux problèmes peuvent être exprimés en terme de contraintes comme les problèmes d'emploi de temps, affecter des stages à des étudiants en respectant leurs souhaits, de gestion de trafic ainsi que certains problèmes de planification et d'optimisation (comme le problème de routage de réseaux de télécommunication). La notion de "Problème à Satisfaction de Contraintes" (ou CSP en abrégé, pour Constraint Satisfaction Problem) désigne l'ensemble de ces problèmes, définis par des contraintes, et consistant à chercher une solution les respectant.

2.2 Notion de contrainte

En mathématiques, une contrainte est une relation logique (une propriété qui doit être vérifiée) entre différentes inconnues, appelées variables, chacune prenant ses valeurs dans un ensemble donné, appelé domaine. Ainsi, une contrainte restreint les valeurs que peuvent prendre simultanément les variables. Par exemple, la contrainte " $x + 3*y = 12$ " restreint les valeurs que l'on peut affecter simultanément aux variables x et y .

2.2.1 Quelques caractéristiques des contraintes

Une contrainte est *relationnelle*: s'elle n'est pas "dirigée" comme une fonction qui définit la valeur d'une variable en fonction des valeurs des autres variables. Ainsi, la contrainte " $x - 2*y = z$ " permet de déterminer z dès lors que x et y sont connues, mais aussi x dès lors que y et z sont connues et y dès lors que x et z sont connues.

Notons qu'une contrainte est *déclarative*: s'elle spécifie quelle relation on doit retrouver entre les variables, sans donner de procédure opérationnelle pour vérifier cette relation.

Notons que *l'ordre dans lequel sont posées les contraintes n'est pas significatif*: la seule chose importante est que toutes les contraintes soient satisfaites (cependant, dans certains langages de programmation par contraintes l'ordre dans lequel les contraintes sont ajoutées peut avoir une influence sur l'efficacité de la résolution).

2.2.2 Définition d'une contrainte en extension ou en intension

Une contrainte est une relation entre différentes variables. Cette relation peut être définie en *extension* ou en *intension* :

- Pour définir une contrainte en extension, on énumère les valeurs appartenant à la relation. Par exemple, si les domaines des variables x et y contiennent les

valeurs 0, 1 et 2, alors on peut définir la contrainte "*x est plus petit que y*" en extension par " $(x=0 \text{ et } y=1) \text{ ou } (x=0 \text{ et } y=2) \text{ ou } (x=1 \text{ et } y=2)$ ", Pour définir une contrainte en intention, on utilise des propriétés mathématiques connues. Par exemple : " $x < y$ ".

2.2.3 Arité d'une contrainte

L'arité d'une contrainte est le nombre de variables sur lesquelles elle porte la contrainte est :

- **unaire** si son arité est égale à 1 (elle porte sur une variable), par exemple " $x * x = 4$ ".
- **binnaire** si son arité est égale à 2 (sur 2 variables), par exemple " $x \neq y$ ".
- **ternaire** si son arité est égale à 3 (sur 3 variables), par exemple " $x + y < 3 * z - 4$ ".
- **n-aire** si son arité est égale à n (elle met en relation un ensemble de n variables). Dans ce cas que la contrainte est globale. Par exemple, une contrainte globale courante (et très pratique) est la contrainte "*toutes Différentes(E)*", où E est un ensemble de variables, qui contraint toutes les variables appartenant à E à prendre des valeurs différentes.

2.2.4 - Différents types de contraintes

On distingue différents types de contraintes en fonction des domaines de valeurs des variables

- Les **contraintes numériques**, portant sur des variables à valeurs numériques : une contrainte numérique est une égalité ($=$) , une différence ($\neq$) ou une inégalité ($<$, $\leq$, $>$, $\geq$) entre 2 expressions arithmétiques.

On distingue :

- les **contraintes numériques linéaires**, quand les expressions arithmétiques sont linéaires, par exemple " $4 * x - 3 * y + 8 * z < 10$ ".
- les **contraintes numériques non linéaires**, quand les expressions arithmétiques contiennent des produits de variables, exemple " $x * y = 4$ " ou " $x * \log(y) = 4$ ".
- Les **contraintes booléennes**, portant sur des variables à valeur booléenne (vrai ou faux) : une contrainte booléenne est une implication ($\Rightarrow$), une équivalence ($\Leftrightarrow$) ou une non équivalence ($\nLeftrightarrow$) entre 2 expressions logiques. Par exemple " $(\text{non } a) \text{ ou } b \Rightarrow c$ ".
- Les **contraintes de Herbrand**, portant sur des variables à valeur dans l'univers de Herbrand : une contrainte de Herbrand est une égalité ($=$) ou diségalité ($\neq$) entre 2 termes (appelés aussi arbres) de l'univers de Herbrand. En particulier, unifier deux termes Prolog revient à poser une contrainte d'égalité entre eux. Par exemple l'unification de " $f(X, Y)$ " avec " $f(g(a), Z)$ " s'exprime par la contrainte " $f(X, Y) = f(g(a), Z)$ ".
- Autres comme les contraintes sur les ensembles.

2.3 Problème à Satisfaction de Contraintes

2.3.1 Définition d'un CSP

Un CSP (Problème de Satisfaction de Contraintes) est un problème modélisé sous la forme d'un ensemble de contraintes posées sur des variables, chacune de ces variables prenant ses valeurs dans un domaine. De façon formelle, on définit un CSP par un triplet (X, D, C) tel que

- $X = \{X_1, X_2, \dots, X_n\}$ est l'ensemble des variables (les inconnues) du problème;
- D est la fonction qui associe à chaque variable X_i son domaine $D(X_i)$, c'est-à-dire l'ensemble des valeurs que peut prendre X_i ;
- $C = \{C_1, C_2, \dots, C_k\}$ est l'ensemble des contraintes. Chaque contrainte C_j est une relation entre certaines variables de X , restreignant les valeurs que peuvent prendre simultanément ces variables.

Par exemple, on peut définir le CSP (X, D, C) suivant : $X = \{a, b, c, d\}$.

- $D(a) = D(b) = D(c) = D(d) = \{0, 1\}$.
- $C = \{a \neq b, c \neq d, a+c < b\}$.

Ce CSP comporte 4 variables a, b, c et d , chacune pouvant prendre 2 valeurs (0 ou 1). Ces variables doivent respecter les contraintes suivantes : a doit être différente de b ; c doit être différente de d et la somme de a et c doit être inférieure à b .

2.3.2 - Solution d'un CSP

Etant donné un CSP (X, D, C) , sa résolution consiste à affecter des valeurs aux variables, de telle sorte que toutes les contraintes soient satisfaites. On introduit pour cela les notations et définitions suivantes :

- On appelle **affectation** le fait d'instancier certaines variables par des valeurs (évidemment prises dans les domaines des variables). On notera $A = \{(X_1, V_1), (X_2, V_2), \dots, (X_r, V_r)\}$ l'affectation qui instancie la variable X_1 par la valeur V_1 , la variable X_2 par la valeur V_2 , ..., et la variable X_r par la valeur V_r .

Par exemple, sur le CSP précédent, $A = \{(b, 0), (c, 1)\}$ est l'affectation qui instancie b à 0 et c à 1.

- Une affectation est dite **totale** si elle instancie toutes les variables du problème ; elle est dite **partielle** si elle n'en instancie qu'une partie.

Sur notre exemple, $A_1 = \{(a, 1), (b, 0), (c, 0), (d, 0)\}$ est une affectation totale ; $A_2 = \{(a, 0), (b, 0)\}$ est une affectation partielle.

- Une affectation A **viole** une contrainte C_k si toutes les variables de C_k sont instanciées dans A , et si la relation définie par C_k n'est pas vérifiée pour les valeurs des variables de C_k définies dans A .

Sur notre exemple, l'affectation partielle $A_2 = \{(a,0),(b,0)\}$ viole la contrainte $a \neq b$; en revanche, elle ne viole pas les deux autres contraintes dans la mesure où certaines de leurs variables ne sont pas instanciées dans A_2 .

- Une affectation (totale ou partielle) est **consistante** si elle ne viole aucune contrainte, et **inconsistante** si elle viole une ou plusieurs contraintes.

Sur notre exemple, l'affectation partielle $\{(c,0),(d,1)\}$ est consistante, tandis que l'affectation partielle $\{(a,0),(b,0)\}$ est inconsistante.

- Une solution est une affectation totale consistante, c'est-à-dire une évaluation de toutes les variables du problème qui ne viole aucune contrainte.

Exemple, $A = \{(a,0),(b,1),(c,0),(d,1)\}$ est une affectation totale consistante: c'est une solution.

2.3.3 Notion de CSP surcontraint ou souscontraint

Lorsqu'un CSP n'a pas de solution, on dit qu'il est *surcontraint* : il y a trop de contraintes et on ne peut pas toutes les satisfaire. Dans ce cas, on peut souhaiter trouver l'affectation totale qui viole le moins de contraintes possibles.

Un tel CSP est appelé *max-CSP* (on cherche à maximiser le nombre de contraintes satisfaites).

Une autre possibilité est d'affecter un poids à chaque contrainte (une valeur proportionnelle à l'importance de cette contrainte, et de chercher l'affectation totale qui minimise la somme des poids des contraintes violées). Un tel CSP est appelé *CSP valué (VCSP)*.

Il existe encore d'autres types de CSPs, appelés *CSPs basés sur les semi-anneaux* (semiring based CSPs), permettant de définir plus finement des préférences entre les contraintes.

Inversement, lorsqu'un CSP admet beaucoup de solutions différentes, on dit qu'il est *sous-contraint*. Si les différentes solutions ne sont pas toutes équivalentes, dans le sens où certaines sont mieux que d'autres, on peut exprimer des préférences entre les différentes solutions. Pour cela, on ajoute une fonction qui associe une valeur numérique à chaque solution, valeur dépendante de la qualité de cette solution. L'objectif est alors de trouver la solution du CSP qui maximise cette fonction. Un tel CSP est appelé *CSOP* (Constraint Satisfaction Optimisation Problem).

2.3.4 Types de CSP

- Le type le plus simple de met en jeu des variables discrets ayant des données finies. C'est le cas des problèmes de coloriage de la carte (voir un exemple à la fin de chapitre).
- Les CSP booléens dont les variables valent soit vrai, soit faux font partie des problèmes CSP à domaines finis tels que les problèmes SAT.

2.3.5 Formulation d'un problème sous la forme d'un CSP

Pour formuler un problème sous la forme d'un CSP, il s'agit tout d'abord d'identifier l'ensemble des variables X (les inconnues du problème), ainsi que la fonction D qui associe à chaque variable de X son domaine (les valeurs que la variable peut prendre). Il faut ensuite identifier les contraintes C entre les variables. Notons qu'à ce niveau, on ne se soucie pas de savoir comment résoudre le problème : on cherche simplement à le spécifier formellement. Cette phase de spécification est indispensable à tout processus de résolution de problème ; les CSPs fournissent un cadre structurant à cette formalisation.

2.3.6 Discussion

La question que l'on peut se poser est : "Quelle est la meilleure modélisation ?"

1. Celle qui modélise le mieux la réalité du problème.
2. Celle qui est la plus facile à trouver. De ce point de vue, la première modélisation est probablement plus "simple".
3. Celle qui permet de résoudre le problème le plus efficacement. On ne peut vraiment répondre à cette question qu'à partir du moment où l'on sait comment un CSP est résolu.

Un même problème peut généralement être modélisé par différents CSPs. On verra plus tard que la modélisation choisie peut avoir une influence sur l'efficacité de la résolution.

2.4 Etude de Cas

2.4.1 Description du problème

Nous intéressons à une carte administrative de l'Australie représentant des états et les régions de ce pays et que nous souhaitons colorier chaque région en rouge, vert et bleu de telle manière qu'aucune région n'ait la même couleur qu'une de ses voisines.



✓ Formulation du problème

Pour formuler cette tâche sous la forme d'un CSP, on définit :

- Les variables destinées à représenter les régions par : AO , TN , Q , NGS , V , AM et T .
- Le domaine de chaque variable est l'ensemble : $\{\text{rouge, vert, bleu}\}$.

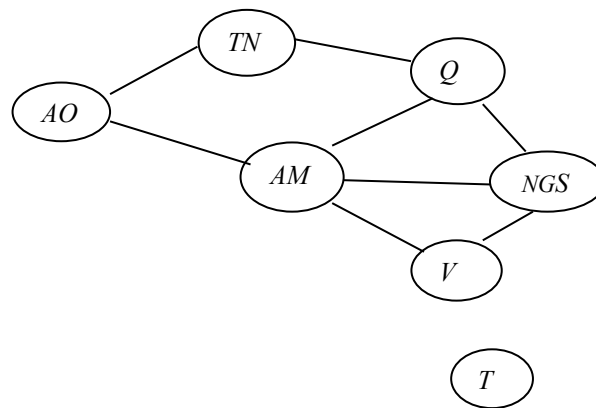
- Les contraintes exigent que des régions voisines aient des couleurs distinctes. Par exemple les combinaisons autorisées pour *AO* et *TN* sont les paires :

(rouge, vert), (rouge, bleu), (vert, rouge), (vert, bleu), (bleu, rouge), (bleu, vert)}

Il existe de nombreuses solutions, par exemple :

{ *AO*=rouge, *TN*=vert, *Q*=rouge, *NGS*=vert, *V*=rouge, *AM*=bleu, *T*=rouge}.

Il est possible de visualiser un CSP sous la forme **d'un graphe de contraintes** qui porte une aide appréciable : Les nœuds du graphe correspondent à des variables du problème et les arcs renvoient à des contraintes.



❖ **Exercices** (Voir Série 2)

Chapitre 3 Modélisation des Problèmes de Satisfaction de Contraintes

3.1 Objectif

Le problème de satisfaction de contraintes, est un problème modélisé sous la forme d'un ensemble de contraintes posées sur des variables, chacune de ces variables prenant ses valeurs dans un domaine. De façon plus formelle, un CSP est défini par un triplet (X, D, C) tel que :

- $X = \{X_1, X_2, \dots, X_n\}$ est l'ensemble des variables (les inconnues) du problème ;
- D est la fonction qui associe à chaque variable X_i son domaine $D(X_i)$, c'est-à-dire l'ensemble des valeurs que peut prendre X_i ;
- $C = \{C_1, C_2, \dots, C_k\}$ est l'ensemble des contraintes. Chaque contrainte C_j est une relation entre certaines variables de X , restreignant les valeurs que peuvent prendre simultanément ces variables.

L'objectif de ce chapitre est de s'entraîner à modéliser un problème sous la forme d'un CSP, en identifiant l'ensemble des variables X et leurs domaines de valeurs D , ainsi que les contraintes C entre les variables. Nous proposons pour cela 2 problèmes différents pour les modéliser sous la forme de CSPs (X, D, C) . Notons que cette phase de modélisation est un préliminaire indispensable à l'utilisation de la programmation par contraintes pour résoudre des problèmes. Les 2 problèmes qui seront modélisés vont être résolus à l'aide de la programmation par contraintes.

3.2 Etapes de Modélisation d'un problème

La modélisation d'un problème sous forme de CSP est complexe, il s'agit tout d'abord d'identifier l'ensemble des variables X (les inconnues du problème), ainsi que la fonction D qui associe à chaque variable de X son domaine (les valeurs qu'elle peut prendre). Il faut ensuite identifier les contraintes C entre les variables. Notons qu'à ce niveau, on ne se soucie pas de savoir comment résoudre le problème : on cherche simplement à le spécifier formellement. Cette phase de spécification est indispensable à tout processus de résolution de problème. Un même problème peut généralement être modélisé par différents CSPs.

Les étapes :

1. Identifier les variables : les inconnues
2. Définir les domaines de valeur de ces variables
3. Identifier ou lister les contraintes définissant le problème.

Souvent plusieurs modélisations possibles, critères de choix :

- simplicité d'expression,
- efficacité : taille de l'espace de recherche de solutions.

3.3 Modélisation des CSPs

Exercice 1 : Retour de monnaie

On s'intéresse à un distributeur automatique de boissons. L'utilisateur insère des pièces de monnaie pour un total de T centimes d'Euros, puis il sélectionne une boisson, dont le prix est de P centimes d'Euros (T et P étant des multiples de 10). Il s'agit alors de calculer la monnaie à rendre, sachant que le distributeur a en réserve E_2 pièces de 2 €, E_1 pièces de 1 €, $C50$ pièces de 50 centimes, $C20$ pièces de 20 centimes et $C10$ pièces de 10 centimes.

1. Modélisez ce problème sous la forme d'un CSP.
2. Comment pourrait-on exprimer le fait que l'on souhaite que le distributeur rende le moins de pièces possibles ?

Solution

1. Modélisation de problème sous la forme d'un CSP.

- Les variables : $X = \{ XE2, XE, XC50, XC20, XC10 \}$
- Les domaines : spécifient que le nombre de pièces retournées, pour un type de pièce donnée, est compris entre 0 et le nombre de pièces de ce type que l'on a en réserve :

$$D(XE2) = \{0, 1, \dots, E2\}$$

$$D(XE1) = \{0, 1, \dots, E1\}$$

$$D(XC50) = \{0, 1, \dots, C50\}$$

$$D(XC20) = \{0, 1, \dots, C20\}$$

$$D(XC10) = \{0, 1, \dots, C10\}$$

- Les contraintes spécifient que la somme à retourner doit être égale à la somme insérée moins le prix à payer :

$$C = \{ 200 * XE2 + 100 * XE1 + 50 * XC50 + 20 * XC20 + 10 * XC10 = T - P \}$$

2. Comment pourrait-on exprimer le fait que l'on souhaite que le distributeur rende le moins de pièces possibles ?

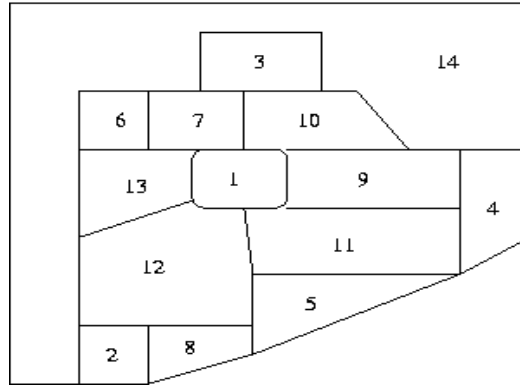
Pour exprimer le fait que l'on souhaite que le distributeur rende le moins de pièces possibles, on pourrait ajouter à ce CSP une fonction :

$$f(X) = XE2 + XE1 + XC50 + XC20 + XC10$$

Dans ce cas, pour résoudre le CSP il suffit de chercher une affectation de X complète et consistante qui minimise la fonction $f(X)$.

Exercice 2 : Coloriage d'une carte

Il s'agit de colorier les 14 régions de la carte ci-dessous, de sorte que deux régions ayant une frontière en commun soient coloriées avec des couleurs différentes. On dispose pour cela des 4 couleurs suivantes : bleu, rouge, jaune et vert.



-Modélisez ce problème sous la forme d'un CSP.

Solution

Formalisation du problème sous la forme d'un CSP (X, D, C)

- Les variables : $X = \{X_1, X_2, \dots, X_{14}\}$
(une variable X_i différente par région i à colorier.)
- Les domaines $D(X_i) = \{ \text{bleu, rouge, vert, jaune} \}$ pour tout X_i élément de X
(Chaque région peut être coloriée avec une des 4 couleurs.)
- Les contraintes $C = \{ X_i \neq X_j / X_i \text{ et } X_j \text{ sont 2 variables différentes de } X \text{ et } \text{voisines}(X_i, X_j) = \text{vrai} \}$

Chapitre 4 Résolution des Problèmes de Satisfaction de Contraintes

4.1 Présentation

Dans ce chapitre, nous allons étudier quelques algorithmes permettant de résoudre, de façon générique, certains CSPs. On se limitera aux CSPs sur les domaines finis, c'est-à-dire, les CSPs dont les domaines des variables sont des ensembles finis de valeurs. Le principe commun à tous les algorithmes est d'explorer méthodiquement l'ensemble des affectations possibles jusqu'à, soit trouver une solution (le CSP est consistant), soit démontrer qu'il n'existe pas de solution (le CSP est inconsistant).

4.2 Résolution de problèmes d'optimisation sous contraintes

Les algorithmes que nous allons étudier permettent de rechercher une solution à un CSP (c'est-à-dire la première que l'on trouve). Suivant les applications, résoudre un CSP peut signifier autre chose que chercher simplement une solution. En particulier, il peut s'agir de chercher la "meilleure" solution selon un critère donné.

Par exemple, pour le cas de problème du coloriage d'une carte, on peut chercher la solution qui utilise le moins de couleurs possibles ; pour le problème du retour de monnaie, on peut chercher la solution qui minimise le nombre total de pièces rendues.

Ces problèmes d'optimisation sous contraintes, où l'on cherche à optimiser une fonction objective donnée tout en satisfaisant toutes les contraintes, peuvent être résolus en explorant l'ensemble des affectations possibles selon la stratégie de "Séparation & Evaluation " ("Branch & Bound").

4.3 Algorithmes de résolution de CSPs autres que sur les domaines finis

Les algorithmes traités dans cette partie, permettent de résoudre de façon générique n'importe quel CSP sur les domaines finis. Il existe d'autres algorithmes plus spécifiques qui tirent parti de connaissances sur les domaines et les types de contraintes pour résoudre des CSPs. Par exemple, les CSPs numériques linéaires sur les réels peuvent être résolus par l'algorithme du Simplex (recherche opérationnelle) ; les CSPs numériques linéaires sur les entiers peuvent être résolus en combinant l'algorithme du Simplex avec une stratégie de "Séparation et Evaluation" ; les CSPs numériques non linéaires sur les réels peuvent être résolus en utilisant des techniques de propagation d'intervalles; etc.

4.4 Algorithmes incomplets pour la résolution de CSPs

Les *algorithmes* sont dits *complets*, dans le sens où l'on est certain de trouver une solution si le CSP est consistant. Cette propriété de complétude est très intéressante dans la mesure où

elle offre des garanties sur la qualité du résultat. En revanche, elle impose de parcourir exhaustivement l'ensemble des combinaisons possibles, et même si l'on utilise différentes techniques et heuristiques pour réduire la combinatoire, il existe certains problèmes pour lesquels ce genre d'algorithme ne termine pas "en un temps raisonnable". Par opposition, les algorithmes incomplets ne cherchent pas à envisager toutes les combinaisons, mais cherchent à trouver le plus vite possible une affectation "acceptable" : ces algorithmes permettent de trouver rapidement de bonnes affectations (qui violent peu ou pas de contraintes) ; en revanche, on n'est pas certain que la meilleure affectation trouvée par ces algorithmes soit effectivement optimale ; on est par ailleurs certain que l'on ne pourra pas prouver l'optimalité de l'affectation trouvée. Il existe différents algorithmes incomplets pour résoudre de façon générique des CSPs sur les domaines finis, généralement basés sur des techniques de recherche locale (éventuellement combinées avec des méthodes Tabou, du Recuit simulé, des algorithmes à base de colonies de fourmis, ...).

4.4.1 Algorithme "génère et teste"

4.4.1.1 Principe de l'algorithme "génère et teste"

La façon la plus simple de résoudre un CSP sur les domaines finis consiste à énumérer toutes les affectations totales possibles jusqu'à en trouver une qui satisfait toutes les contraintes. Ce principe est repris dans la fonction récursive "*genereEtTeste(A,(X,D,C))*" décrite ci-dessous. Dans cette fonction, *A* contient une affectation partielle et *(X,D,C)* décrit le CSP à résoudre (au premier appel de cette fonction, l'affectation partielle *A* sera vide). La fonction retourne *vrai* si on peut étendre l'affectation partielle *A* en une affectation totale consistante (une solution), et faux sinon.

fonction *genereEtTeste(A,(X,D,C))* retourne un booléen

Précondition :

(X,D,C) = un CSP sur les domaines finis
A = une affectation partielle pour *(X, D,C)*

Postrelation :

retourne vrai si l'affectation partielle *A* peut être étendue en une solution pour *(X,D,C)*, faux sinon

début

si toutes les variables de *X* sont affectées à une valeur dans *A* **alors**

/* *A* est une affectation totale */

si *A* est consistante **alors**

/* *A* est une solution */

retourner vrai

sinon

retourner faux

fin

sinon /* A est une affectation partielle */

choisir une variable X_i de X qui n'est pas encore affectée à une valeur dans A

pour toute valeur V_i appartenant à $D(X_i)$ **faire**

si $\text{genereEtTeste}(A \cup \{(X_i, V_i)\}, (X, D, C)) = \text{vrai}$ **alors** retourner vrai

finpour

retourner faux

fin

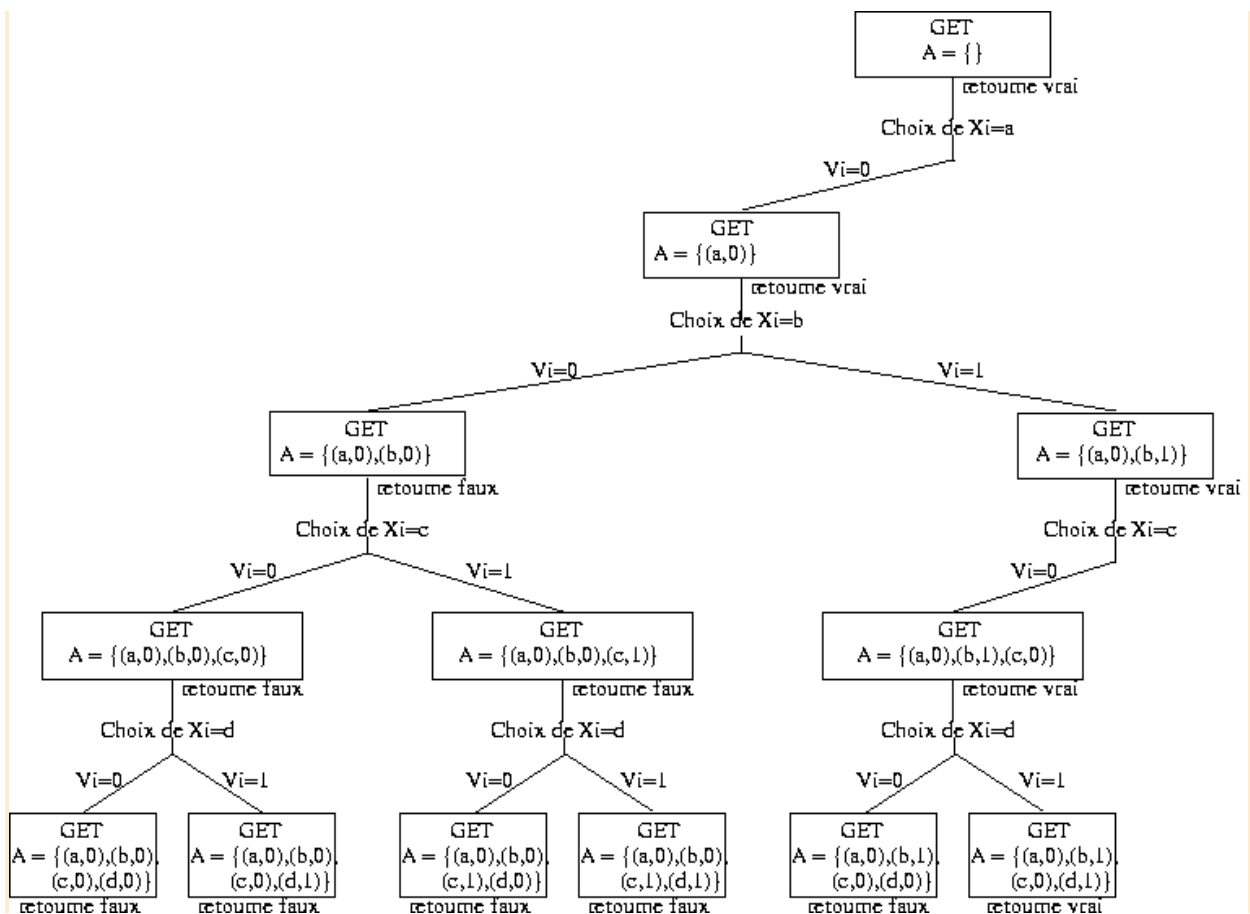
fin

4.4.1.2 Exemple de trace d'exécution de "génère et teste"

Considérons par exemple le CSP (X, D, C) suivant :

- $X = \{a, b, c, d\}$
- $D(a) = D(b) = D(c) = D(d) = \{0, 1\}$
- $C = \{a \neq b, c \neq d, a + c < b\}$

L'enchaînement des appels successifs à la fonction genereEtTeste (abrégée par GET) est représenté ci-dessous (chaque rectangle correspond à un appel de la fonction, et précise la valeur de l'affectation partielle en cours de construction A).



4.4.1.3 Analyse critique de "génère et teste" et notion d'espace de recherche d'un CSP

L'algorithme "génère et teste" que nous venons de voir énumère l'ensemble des affectations complètes possibles, jusqu'à en trouver une qui soit consistante. L'ensemble des affectations complètes est appelé l'espace de recherche du CSP.

Première Limite : Si le domaine de certaines variables contient une infinité de valeurs, alors cet espace de recherche est infini et on ne pourra pas énumérer ses éléments en un temps fini. Néanmoins, même en se limitant à des domaines comportant un nombre fini de valeurs, l'espace de recherche est souvent de taille tellement importante que l'algorithme "génère et teste" ne pourra se terminer en un temps "raisonnable".

En effet, l'espace de recherche d'un CSP (X, D, C) comportant n variables ($X = \{X_1, X_2, \dots, X_n\}$) est défini par

$$E = \{ \{ (X_1, v_1), (X_2, v_2), \dots, (X_n, v_n) \} \mid \text{quelque soit } i \text{ compris entre } 1 \text{ et } n, v_i \text{ est un élément de } D(X_i) \}$$

et le nombre d'éléments de cet espace de recherche est défini par

$$|E| = |D(X_1)| * |D(X_2)| * \dots * |D(X_n)|$$

de sorte que, si tous les domaines des variables sont de la même taille k ($|D(X_i)| = k$), alors la taille de l'espace de recherche est $|E| = k^n$.

Ainsi, le nombre d'affectations à générer croît de façon exponentielle en fonction du nombre de variables du problème. Considérons plus précisément un CSP ayant n variables, chaque variable pouvant prendre 2 valeurs ($k=2$). Dans ce cas, le nombre d'affectations totales possibles pour ce CSP est 2^n .

Nombre de variables n	Nombre d'affectations totales 2^n	Temps estimé
10	environ 10^3	environ 1 millionième de seconde
20	environ 10^6	environ 1 millième de seconde
30	environ 10^9	environ 1 seconde
40	environ 10^{12}	environ 16 minutes
50	environ 10^{15}	environ 11 jours
60	environ 10^{18}	environ 32 ans

Nous allons étudier dans la suite les améliorations de "génère et teste". Dès lors que le CSP a plus d'une trentaine de variables, on ne peut pas appliquer l'algorithme "génère et teste". Il faut donc chercher à réduire l'espace de recherche. Pour cela, on peut notamment :

- ne développer que les affectations partielles consistantes : dès lors qu'une affectation partielle est inconsistante, il est inutile de chercher à l'étendre en une affectation totale puisque celle-ci sera nécessairement inconsistante ;

Cette idée est reprise dans l'algorithme "simple retour-arrière" que nous étudierons par la suite.

- réduire les tailles des domaines des variables en leur enlevant les valeurs "incompatibles" : pendant la génération d'affectations, on filtre le domaine des variables pour ne garder que les valeurs "localement consistantes" avec l'affectation en cours de construction, et dès lors que le domaine d'une variable devient vide, on arrête l'énumération pour cette affectation partielle ;

Cette idée est reprise dans l'algorithme "anticipation" que nous allons traiter au 3ème point de ce chapitre.

- introduire des "heuristiques" pour "guider" la recherche : lorsqu'on énumère les affectations possibles, on peut essayer d'énumérer en premier celles qui sont les plus "prometteuses", en espérant ainsi tomber rapidement sur une solution.

Cette idée est reprise dans le point 4 de ce chapitre.

Il existe encore bien d'autres façons de tenter de réduire la combinatoire, afin de rendre l'exploration exhaustive de l'espace de recherche possible que nous ne verrons pas dans ce cours. Par exemple, lors d'un échec, on peut essayer d'identifier la cause de l'échec (quelle est la variable qui viole une contrainte) pour ensuite "retourner en arrière" directement là où cette variable a été instanciée afin de remettre en cause plus rapidement la variable à l'origine de l'échec. C'est ce que l'on appelle le "retour arrière intelligent" ("intelligent backtracking").

Une autre approche particulièrement séduisante consiste à exploiter des connaissances sur les types de contraintes utilisées pour réduire l'espace de recherche.

4.4.2 Algorithme "simple retour-arrière"

4.4.2.1 Principe de l'algorithme "simple retour-arrière"

Une première façon d'améliorer l'algorithme "génère et teste" consiste à tester au fur et à mesure de la construction de l'affectation partielle sa consistance : dès lors qu'une affectation partielle est inconsistante, il est inutile de chercher à la compléter. Dans ce cas, on "retourne en arrière" ("backtrack" en anglais) jusqu'à la plus récente instanciation partielle consistante que l'on peut étendre en affectant une autre valeur à la dernière variable affectée. Par exemple, sur la trace d'exécution décrite en 1.2, on remarque que l'algorithme génère tous les prolongements de l'affectation partielle $A = \{(a,0), (b,0)\}$, en énumérant toutes les possibilités d'affectation pour les variables c et d, alors qu'elle viole la contrainte $a \neq b$. L'algorithme "simple retour-arrière" ne va donc pas chercher à étendre cette affectation, mais va "retourner en arrière" à l'affectation partielle précédente $A = \{(a,0)\}$, et va l'étendre en affectant 1 à b, etc.

Ce principe est repris dans la fonction récursive "simpleRetourArrière(A,(X,D,C))" décrite ci-dessous. Dans cette fonction, A contient une affectation partielle et (X,D,C) décrit le CSP à résoudre (au premier appel de cette fonction, l'affectation partielle A sera vide). La fonction retourne *vrai* si on peut étendre l'affectation partielle A en une affectation totale consistante (une solution), et faux sinon.

fonction simpleRetourArrière(A,(X,D,C)) **retourne** un booléen

Précondition :

A = affectation partielle

(X,D,C) = un CSP sur les domaines finis

Postrelation :

retourne vrai si A peut être étendue en une solution pour (X,D,C) , faux sinon

début

si A n'est pas consistante **alors retourner** faux

finsi

si toutes les variables de X sont affectées à une valeur dans A **alors**

/ A est une affectation totale et consistante = une solution */*

retourner vrai

Sinon */* A est une affectation partielle consistante */*

choisir une variable X_i de X qui n'est pas encore affectée à une valeur dans A

pour toute valeur V_i appartenant à $D(X_i)$ **faire**

si simpleRetourArrière($A \cup \{(X_i, V_i)\}$, (X,D,C)) = vrai **alors retourner** vrai

finpour

retourner faux

finsi

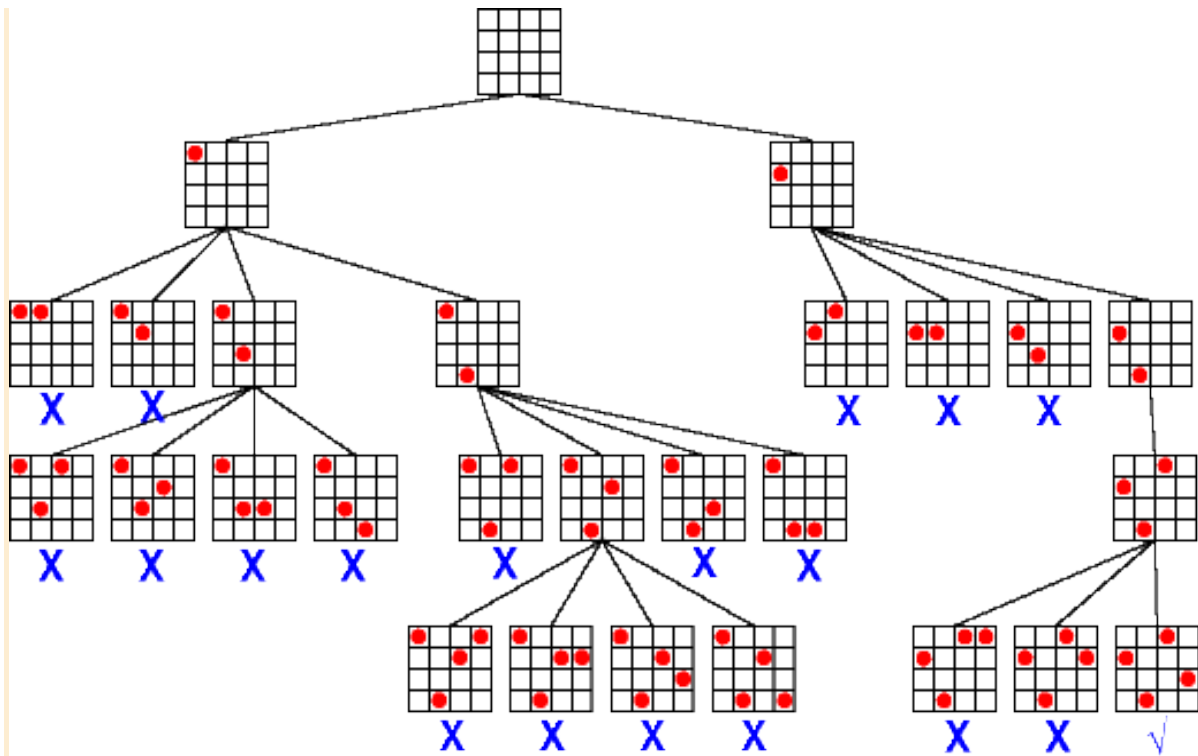
fin

4.4.2.2 Exemple de "trace d'exécution" de SimpleRetourArrière

Considérons le problème du placement de 4 reines sur un échiquier 4x4, et sa modélisation proposée :

- Variables : $X = \{X1, X2, X3, X4\}$
- Domaines : $D(X1) = D(X2) = D(X3) = D(X4) = \{1, 2, 3, 4\}$
- Contraintes : $C = \{X_i \neq X_j / i \text{ élément_de } \{1, 2, 3, 4\}, j \text{ élément_de } \{1, 2, 3, 4\} \text{ et } i \neq j\}$
 $U \{X_{i+i} \neq X_{j+j} / i \text{ élément_de } \{1, 2, 3, 4\}, j \text{ élément_de } \{1, 2, 3, 4\} \text{ et } i \neq j\}$
 $U \{X_{i-i} \neq X_{j-j} / i \text{ élément_de } \{1, 2, 3, 4\}, j \text{ élément_de } \{1, 2, 3, 4\} \text{ et } i \neq j\}$

L'enchaînement des appels successifs à la fonction *SimpleRetourArrière* peut être représenté par l'arbre suivant (chaque nœud correspond à un appel de la fonction ; l'échiquier dessiné à chaque nœud décrit l'affectation partielle en cours) :



4.4.3 Algorithme "anticipation"

4.4.3.1 Notions de filtrage et de consistance locale

Pour améliorer l'algorithme "simple retour-arrière", on peut tenter d'anticiper ("look ahead" en anglais) les conséquences de l'affectation partielle en cours de construction sur les domaines des variables qui ne sont pas encore affectées : si on se rend compte qu'une variable non affectée X_i n'a plus de valeur dans son domaine $D(X_i)$ qui soit "localement consistante" avec l'affectation partielle en cours de construction, alors il n'est pas nécessaire de continuer à développer cette branche, et on peut tout de suite retourner en arrière pour explorer d'autres possibilités.

Pour mettre ce principe en œuvre, on va, à chaque étape de la recherche, filtrer les domaines des variables non affectées en enlevant les valeurs "localement inconsistantes", c'est-à-dire celles dont on peut inférer qu'elles n'appartiendront à aucune solution. On peut effectuer différents filtrages, correspondant à différents niveaux de consistances locales, qui vont réduire plus ou moins les domaines des variables, mais qui prendront aussi plus ou moins de temps à s'exécuter : considérons un CSP (X, D, C) , et une affectation partielle consistante A ,

- le filtrage le plus simple consiste à anticiper d'une étape l'énumération : pour chaque variable X_i non affectée dans A , on enlève de $D(X_i)$ toute valeur v telle que l'affectation $A \cup \{(X_i, v)\}$ soit inconsistante.

Exemple 1

Pour le problème des 4 reines, après avoir instancié X_1 à 1, on peut enlever du domaine de X_2 la valeur 1 (qui viole la contrainte $X_1 \neq X_2$) et la valeur 2 (qui viole la contrainte $1 - X_1 \neq 2 - X_2$).

Ce filtrage permet d'établir ce qu'on appelle la **consistance de noeud**, aussi appelée **1-consistance**.

- un filtrage plus fort, mais aussi plus long à effectuer, consiste à anticiper de deux étapes l'énumération : pour chaque variable X_i non affectée dans A , on enlève de $D(X_i)$ toute valeur v telle qu'il existe une variable X_j non affectée pour laquelle, pour toute valeur w de $D(X_j)$, l'affectation $A \cup \{(X_i, v), (X_j, w)\}$ soit inconsistante.

Exemple 2

Cas de problème des 4 reines, après avoir instancié X_1 à 1, on peut enlever la valeur 3 du domaine de X_2 car si $X_1=1$ et $X_2=3$, alors la variable X_3 ne peut plus prendre de valeurs : si $X_3=1$, on viole la contrainte $X_3 \neq X_1$; si $X_3=2$, on viole la contrainte $X_3+3 \neq X_2+2$; si $X_3=3$, on viole la contrainte $X_3 \neq X_2$; et si $X_3=4$, on viole la contrainte $X_3-3 \neq X_2-2$.

Notons que ce filtrage doit être répété jusqu'à ce qu'aucun domaine ne puisse être réduit.

Ce filtrage permet d'établir ce qu'on appelle la **consistance d'arc**, aussi appelée **2-consistance**.

- un filtrage encore plus fort, mais aussi encore plus long à effectuer, consiste à anticiper de trois étapes l'énumération. Ce filtrage permet d'établir ce qu'on appelle la **consistance de chemin**, aussi appelée **3-consistance**.
- Notons que s'il reste k variables à affecter, et si l'on anticipe de k étapes l'énumération pour établir la k -consistance, l'opération de filtrage revient à résoudre le CSP, c'est-à-dire que toutes les valeurs restant dans les domaines des variables après un tel filtrage appartiennent à une solution.

4.4.3.2 Principe de l'algorithme "anticipation"

Le principe général de l'algorithme "anticipation" reprend celui de l'algorithme "simple retour-arrière", en ajoutant simplement une étape de filtrage à chaque fois qu'une valeur est affectée à une variable. On peut effectuer différents filtrages plus ou moins forts, permettant d'établir différents niveaux de consistance locale (noeud, arc, chemin). Par exemple, la fonction récursive "anticipation/noeud($A, (X, D, C)$)" décrite ci-dessous effectue un filtrage

simple qui établit à chaque étape la consistance de nœud. Dans cette fonction, A contient une affectation partielle consistante et (X,D,C) décrit le CSP à résoudre (au premier appel de cette fonction, l'affectation partielle A sera vide). La fonction retourne *vrai* si on peut étendre l'affectation partielle A en une affectation totale consistante (une solution), et faux sinon.

fonction anticipation/noeud($A,(X,D,C)$) **retourne** un booléen

Précondition :

A = affectation partielle consistante
 (X,D,C) = un CSP sur les domaines finis

Postrelation :

retourne vrai si A peut être étendue en une solution pour (X,D,C) , faux sinon

début

si toutes les variables de X sont affectées à une valeur dans A **alors**

/ A est une affectation totale et consistante = une solution */*

retourner vrai

sinon */* A est une affectation partielle consistante */*

choisir une variable X_i de X qui n'est pas encore affectée à une valeur dans A

pour toute valeur V_i appartenant à $D(X_i)$ **faire**

/ filtrage des domaines par rapport à $A \cup \{(X_i, V_i)\}$ */*

pour toute variable X_j de X qui n'est pas encore affectée **faire**

$D_{filtré}(X_j) \leftarrow \{ V_j \text{ élément de } D(X_j) / A \cup \{(X_i, V_i), (X_j, V_j)\} \text{ est consistante} \}$

si $D_{filtré}(X_j)$ est vide **alors retourner** faux

finpour

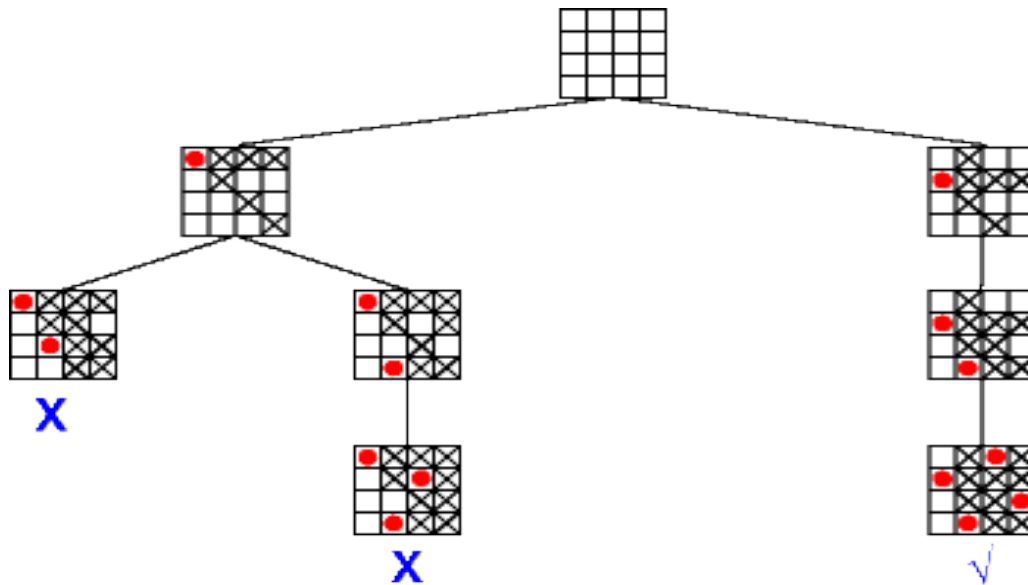
si anticipation($A \cup \{(X_i, V_i)\}, (X, D_{filtré}, C)$)=vrai **alors retourner** vrai

finpour

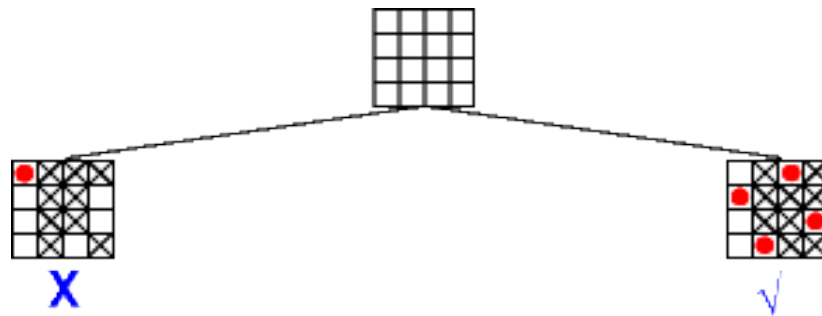
retourner faux

finsi

fin



Si on appliquait un filtrage plus fort, qui rétablirait à chaque étape la consistance d'arc, l'enchainement des appels successifs à la fonction "Anticipation/arc" correspondante serait le suivant (les valeurs supprimées par le filtrage sont marquées d'une croix) :



Ainsi, on constate sur le problème des 4 reines que le filtrage des domaines permet de réduire le nombre d'appels récurrents : on passe de 27 appels pour "simple retour-arrière" à 8 appels pour l'algorithme d'anticipation avec filtrage simple établissant une consistance de nœud. En utilisant des filtres plus forts, comme celui qui établit la consistance d'arc, on peut encore réduire la combinatoire de 8 à 3 appels récurrents. Cependant, il faut noter que plus le filtrage utilisé est fort, plus cela prendra de temps pour l'exécuter.

De façon générale, on constate que, quel que soit le problème considéré, l'algorithme "anticipation/nœud" est généralement plus rapide que l'algorithme "simple retour-arrière" car le filtrage utilisé est vraiment peu coûteux. En revanche, si l'algorithme "anticipation/arc" envisage toujours moins de combinaisons que l'algorithme "anticipation/nœud", il peut parfois prendre plus de temps à l'exécution car le filtrage pour établir une consistance d'arc est plus long que celui pour établir la consistance de nœud.

4.4.4 Intégration d'heuristiques

Les algorithmes que nous venons d'étudier choisissent, à chaque étape, la prochaine variable à instancier parmi l'ensemble des variables qui ne sont pas encore instanciées ; ensuite, une fois la variable choisie, ils essaient de l'instancier avec les différentes valeurs de son domaine. Ces algorithmes ne disent rien sur l'ordre dans lequel on doit instancier les variables, ni sur l'ordre dans lequel on doit affecter les valeurs aux variables. Ces deux ordres peuvent changer considérablement l'efficacité de ces algorithmes. Malheureusement, le problème général de la satisfaction d'un CSP sur les domaines finis étant NP-complet. En revanche, on peut intégrer des heuristiques pour déterminer l'ordre dans lequel les variables et les valeurs doivent être considérées : une heuristique est une règle non systématique (dans le sens où elle n'est pas fiable à 100%) qui nous donne des indications sur la direction à prendre dans l'arbre de recherche.

Les heuristiques concernant l'ordre d'instanciation des valeurs sont généralement dépendantes de l'application considérée et difficilement généralisables. En revanche, il existe de nombreuses heuristiques d'ordre d'instanciation des variables qui permettent bien souvent d'accélérer considérablement la recherche. L'idée générale consiste à instancier en premier les variables les plus "critiques", c'est-à-dire celles qui interviennent dans beaucoup de contraintes et/ou qui ne peuvent prendre que très peu de valeurs. L'ordre d'instanciation des variables peut être :

- **statique**, quand il est fixé avant de commencer la recherche ;
Par exemple, on peut ordonner les variables en fonction du nombre de contraintes portant sur elles : l'idée est d'instancier en premier les variables les plus contraintes, c'est-à-dire celles qui participent au plus grand nombre de contraintes.
- ou **dynamique**, quand la prochaine variable à instancier est choisie dynamiquement à chaque étape de la recherche.

Par exemple, l'heuristique "échec d'abord" ("first-fail" en anglais) consiste à choisir, à chaque étape, la variable dont le domaine a le plus petit nombre de valeurs localement consistantes avec l'affectation partielle en cours. Cette heuristique est généralement couplée avec l'algorithme "anticipation", qui filtre les domaines des variables à chaque étape de la recherche pour ne garder que les valeurs qui satisfont un certain niveau de consistance locale.

Exercices (Série 3).